



How RescueRoute works

A friendly walk through the graphs, heaps and trees that send the right ambulance down the right road in Bengaluru. No prior project experience needed: if you know arrays, loops and recursion, you can follow every page.

What's inside

- | | | | |
|---|----------------------------------|----|--------------------|
| 1 | What the prototype does | 8 | Hospital coverage |
| 2 | Big-O in two minutes | 9 | KD-tree |
| 3 | Graphs: turning a city into data | 10 | Union-find |
| 4 | Heaps and priority queues | 11 | LRU cache |
| 5 | Dijkstra's shortest path | 12 | Small helpers |
| 6 | A* search | 13 | Exploring the code |
| 7 | Choosing the ambulance | 14 | Glossary |

What the prototype does

RescueRoute is a pretend emergency control room for Bengaluru. Calls come in, ambulances get sent, roads close, traffic builds up, and the system has to keep making good decisions, fast.

Open the live prototype and you'll see a map of the real city. Coloured dots are emergency calls. Little pills with numbers are ambulances (there are 14). Red-cross squares are hospitals. Everything moves on its own because a **simulation clock** keeps ticking: new calls appear every few simulated minutes, and ambulances drive along roads in real time.

The life of one emergency call

Follow a single call from start to finish and you'll touch almost every data structure in the project.

- 1 **Someone reports a call.** You click a street on the map. The app finds the nearest real intersection using a **KD-tree** (chapter 9) and names the place after the closest bus stop.
- 2 **The call joins a queue.** If no ambulance is free, it waits in a **priority queue** (chapter 4) where critical calls always come out before minor ones.
- 3 **The best ambulance is picked.** One clever run of **Dijkstra's algorithm** backwards from the patient finds which free ambulance can get there fastest by road (chapters 5 and 7).
- 4 **The ambulance drives.** Its position along the route is found with **binary search** (chapter 12). If a road on its route closes, it re-plans using **A*** (chapter 6).
- 5 **Patient goes to hospital.** Another search finds the nearest hospital by road. The **coverage map** uses one more Dijkstra variant to show which areas are far from any hospital (chapter 8).

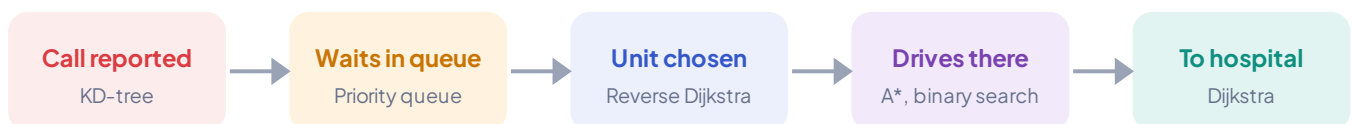


Figure 1. Each stage of a call and the data structure that powers it.

WHERE THE MAP COMES FROM

The roads are real. They were built from public BMTC bus-route data: every bus route traces real Bengaluru roads, so joining them gives a road network of **16,825 intersections**. Ward boundaries come from BBMP open data, and the 14 hospitals are real hospitals the buses stop at.

Big-O in two minutes

Big-O describes how the work grows as the input grows. It's the language we use to say "this approach is fast enough for a whole city" or "this one would take forever".

We write n for the input size. For graphs we use V for the number of vertices (intersections) and E for the number of edges (roads). In RescueRoute, $V = 16,825$ and $E = 64,862$.

$O(1)$

Constant. Same time no matter the size. Looking up a key in a hash map.

$O(\log n)$

Halving each step. Binary search, heap push and pop.

$O(n)$

Look at everything once. Scanning an array.

$O(n \log n)$

Sorting. Building a KD-tree.

Why it matters here

Suppose you want the distance from every intersection to every other one. The classic all-pairs algorithm, Floyd-Warshall, is $O(V^3)$. With 16,825 intersections that is about **4.8 trillion** steps. Dijkstra from a single start is $O((V + E) \log V)$, roughly $81,687 \times 14 \approx$ **1.1 million** steps. That difference is why the app answers in milliseconds instead of hours.

Approach	Complexity	Rough steps for Bengaluru
Floyd-Warshall (all pairs)	$O(V^3)$	4,763,000,000,000
Dijkstra (one start)	$O((V+E) \log V)$	$\approx 1,150,000$
Heap pop (one call)	$O(\log V)$	≈ 14

REMEMBER

Big-O ignores constant factors and small inputs. It tells you which approach *scales*. For a city-sized graph, scaling is everything.

Graphs: turning a city into data

A graph is just a set of points (vertices) joined by lines (edges). Road maps are graphs in disguise.

Vertex: an intersection or bend in a road. Each one has a latitude and longitude.

Edge: a stretch of road from one vertex to the next.

Directed: an edge has a direction. A two-way road is stored as two edges, one each way. A one-way street is one edge.

Weighted: every edge has a cost. In RescueRoute the cost is **travel time in seconds**, not distance.

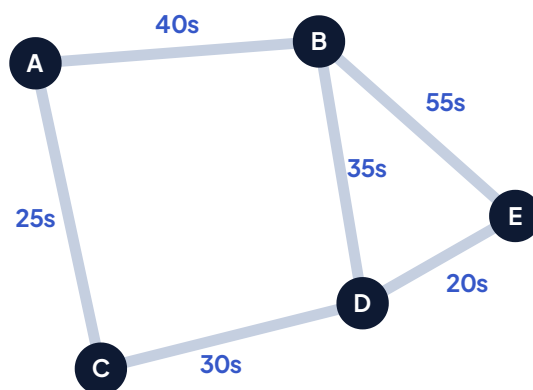


Figure 2. Five intersections, six roads, each labelled with its travel time.

The edge weight formula

```
cost(edge) = length_in_metres / speed_in_metres_per_second * traffic_factor
// closed road    → cost = Infinity (no search will ever use it)
// rush hour      → traffic_factor goes from 1.0 up to about 3.5
```

Using time as the weight is the key design choice. Traffic and road closures don't need special code anywhere else: they just change a weight, and every algorithm automatically takes the new situation into account.

Adjacency list vs adjacency matrix

There are two classic ways to store a graph. A **matrix** is a $V \times V$ table where cell $[i][j]$ holds the cost from i to j . An **adjacency list** stores, for each vertex, only the roads that actually leave it.

Storage	Cells needed for Bengaluru	Find neighbours of v
Adjacency matrix	$16,825^2 = 283,080,625$	$O(V)$: scan a whole row
Adjacency list	$\approx 16,825 + 64,862$	$O(\text{degree})$: usually 2–4 roads

Road networks are **sparse**: an intersection connects to only a few others. A matrix would be 99.98% empty, so RescueRoute uses adjacency lists. It actually keeps **two**: **out[v]** (roads leaving v) and **in[v]** (roads arriving at v). The second list lets us search the graph backwards, which chapter 7 shows is a neat trick.

```
// server/graph/Graph.js (simplified)
class Graph {
  lat[], lng[]           // position of each vertex
  from[], to[]           // edge e goes from[e] → to[e]
  length[], speed[], traffic[], blocked[]
  out[v] = [edge ids leaving v]
  in[v]  = [edge ids arriving at v]
}
```

In the code: `server/graph/Graph.js`

Heaps and priority queues

A priority queue is a "to-do list" where you always take out the most important item next. A binary heap is the usual way to build one.

The binary min-heap

A min-heap is a complete binary tree where every parent is smaller than (or equal to) its children. So the smallest item is always at the root. The neat part: we store the tree in a plain **array**, with no pointers.

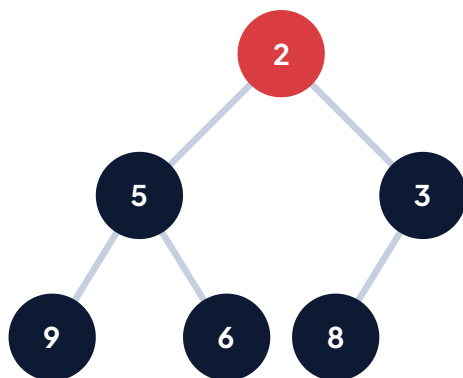


Figure 3. A min-heap. The smallest value, 2, sits at the root.

The same heap as an array:

i	0	1	2	3	4	5
a[i]	2	5	3	9	6	8

For the item at index **i** :

- parent is at $(i - 1) / 2$ (rounded down)
- children are at $2i + 1$ and $2i + 2$

The two moves: sift up and sift down

- **Push**: put the new item at the end of the array, then **sift up**: swap it with its parent while it is smaller. At most one swap per level, so $O(\log n)$.
- **Pop**: take the root, move the last item to the root, then **sift down**: swap it with its smaller child while it is bigger. Also $O(\log n)$.

```
function push(x):
    a.append(x); i = a.length - 1
    while i > 0 and a[parent(i)] > a[i]:
        swap(a[i], a[parent(i)]); i = parent(i)
```

Use 1: the triage queue

When all ambulances are busy, calls wait in a priority queue. The comparison function says "lower severity number first (1 = critical), and if equal, the earlier call first".

```
compare(a, b) = (a.severity - b.severity) || (a.reportedAt - b.reportedAt)
```

So a critical call at 08:20 is served before a minor call at 08:01, even though the minor call waited longer. A normal FIFO queue would get that wrong.

In the code: `server/ds/PriorityQueue.js`

Use 2: the indexed heap inside Dijkstra

Dijkstra (next chapter) often discovers a *shorter* distance to a vertex that is already in the heap. It needs to **decrease its key**. A plain heap can't find the vertex quickly, so RescueRoute keeps an extra array `pos[v]` that remembers where each vertex sits in the heap. Every swap updates `pos`. Now decrease-key is: look up the position, change the value, sift up, all in $O(\log V)$.

WHY BOTHER?

The lazy alternative pushes a duplicate entry every time a distance improves and skips stale ones when popped. That works, but the heap can grow much bigger than V . The indexed heap never holds more than one entry per vertex.

In the code: `server/ds/IndexedMinHeap.js`

CHAPTER 5

Dijkstra's shortest path

Dijkstra's algorithm finds the cheapest route from one start vertex to every other vertex, as long as no edge has a negative cost. Travel times are never negative, so it fits roads perfectly.

The intuition: spreading water

Imagine pouring water at the start intersection. It spreads along roads at a steady speed. The order in which intersections get wet is exactly the order in which Dijkstra finalises them: closest first. Once an intersection is wet, nothing can reach it any faster later, so its distance is final.

The algorithm

```
dist[all] = ∞ ; dist[start] = 0
heap.push(start, 0)
while heap not empty:
    u = heap.popMin()           // closest unfinished vertex
    if u == target: break      // early exit: we already have the answer
    for each road e from u to v:
        newDist = dist[u] + cost(e)
        if newDist < dist[v]:   // "relaxation"
            dist[v] = newDist
            parent[v] = u
            heap.pushOrDecrease(v, newDist)
```

To get the actual route, start at the target and follow `parent` back to the start, then reverse the list.

Worked example

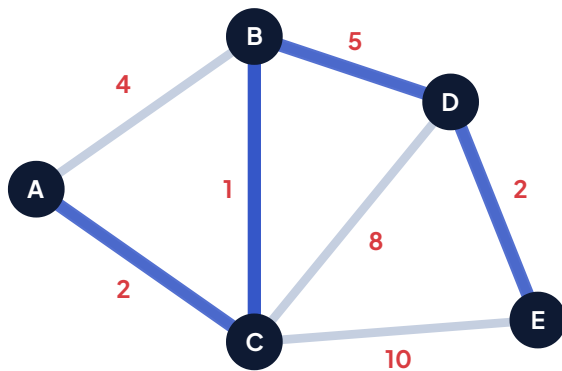


Figure 4. Find the cheapest route from A to E. Blue is the answer.

Pop	A	B	C	D	E
start	0	∞	∞	∞	∞
A (0)	✓	4	2	∞	∞
C (2)	✓	3	✓	10	12
B (3)	✓	✓	✓	8	12
D (8)	✓	✓	✓	✓	10
E (10)	✓	✓	✓	✓	✓

Notice B: first reached at cost 4 directly, then improved to 3 through C. That improvement is a **decrease-key**. Final route: A → C → B → D → E, cost $2 + 1 + 5 + 2 = 10$.

WATCH OUT

Dijkstra assumes costs are never negative. If a road could have a negative cost, a vertex marked "finished" might later get cheaper, and the algorithm would give wrong answers. For that case you'd use Bellman-Ford. RescueRoute's tests actually use Bellman-Ford as a slow-but-simple reference to check that Dijkstra is correct.

Complexity: each vertex is popped once (V pops) and each edge relaxed once (E decrease-keys), each costing $O(\log V)$. Total $O((V + E) \log V)$.

In the code: `server/algorithms/search.js, function search()`

CHAPTER 6

A* search: Dijkstra with a sense of direction

Dijkstra spreads out evenly in every direction, even away from the destination. A* adds a hint about where the goal is, so it explores mostly towards it.

The one change

Dijkstra orders the heap by $g(v)$, the cost so far. A* orders it by

```
f(v) = g(v) + h(v)
// g(v): known travel time from start to v
// h(v): an ESTIMATE of the remaining time from v to the goal
```

In RescueRoute, $h(v)$ = straight-line distance from v to the goal $\div$ the fastest speed on any road. That's the time you'd need if you could fly straight there at top speed.

Why the answer is still optimal

A real drive can never beat flying in a straight line at top speed. So $h(v)$ never *overestimates* the true remaining time. A heuristic with this property is called **admissible**, and with an admissible heuristic A^* is guaranteed to return the shortest route, exactly like Dijkstra.

ANALOGY

You're in a new city walking to a landmark you can see on the skyline. You don't wander down streets heading the opposite way; you prefer streets that point towards it. That's A^* .

How much it saves

The Route Lab view runs both algorithms on the same query. On a cross-city trip from Kempegowda Bus Station to Koramangala (17.7 minutes, 9.1 km):



intersections explored before reaching the goal · both found the same 17.7-minute route

Figure 5. A^* explored 76% fewer intersections for the identical answer.

That's why ambulances use A^* to **re-route** when a road closes: it needs to be fast, and it must still find the best route.

TRY IT

Open Route Lab and pick two places close together, then two far apart. Compare how many intersections each algorithm explores. The gap grows with distance.

In the code: `astar()` in `server/algorithms/search.js`, `heuristic` in `Graph.heuristic()`

CHAPTER 7

Choosing the ambulance: search backwards

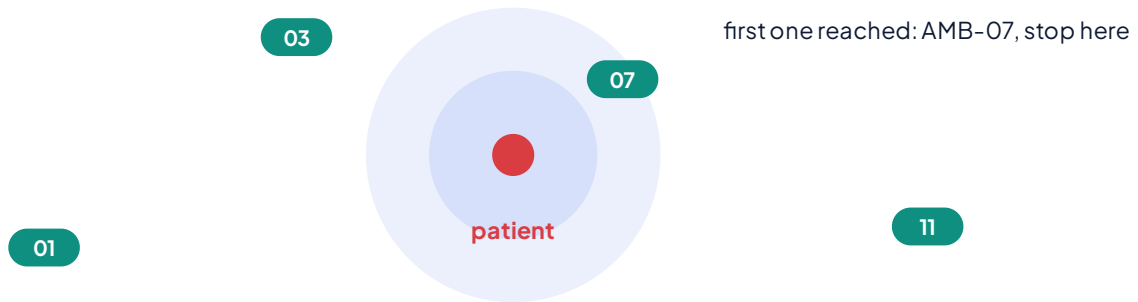
Fourteen ambulances, one patient. Which unit gets there first by road? The obvious way does fourteen searches. RescueRoute does one.

The naive idea

Run Dijkstra from each free ambulance to the patient and pick the smallest time. With k free units that's k searches: $O(k \cdot (V + E) \log V)$.

The trick: reverse the roads

Instead, start one search **at the patient** and walk the roads **backwards**, using the `in[v]` lists from chapter 3. Put every free ambulance's position in a "targets" set. The first target the search pops is the ambulance with the shortest drive to the patient, so we stop right there.



The search grows outward from the patient. Units outside the shaded area are never even looked at.

Figure 6. One reverse search, early exit at the first ambulance found.

Why "backwards" and not just "from the patient"?

Because of one-way streets. The drive from ambulance to patient can be very different from patient to ambulance. Walking incoming edges measures the correct direction: the time for the ambulance to come to the patient.

SEE IT LIVE

The Command Center sometimes logs "Straight-line nearest was AMB-07, but AMB-10 is faster by road". That's this algorithm beating the naïve "closest on the map" guess.

After treatment, the same idea runs **forwards** from the patient with every hospital as a target, so the first hospital reached is the closest by road.

In the code: `_dispatchPending()` and `_sendToHospital()` in `server/sim/Simulation.js`

CHAPTER 8

Hospital coverage: many starts, one search

The Coverage view colours every ward by how long it takes to drive to the nearest hospital. That needs a distance for all 16,825 intersections, from the nearest of 14 hospitals.

Multi-source Dijkstra

Put **all 14 hospitals** in the heap at distance 0 before the search starts, then run Dijkstra normally. Each intersection is reached first by whichever hospital is closest, so one run gives every intersection its "time to nearest hospital".

THINK OF IT AS

A pretend "super hospital" connected to all 14 real hospitals by roads that cost 0. One Dijkstra from the super hospital is exactly the same as a multi-source Dijkstra.

Cost: one search, $O((V + E) \log V)$. In the browser it runs in about **24 milliseconds**. Compare that with the 4.8 trillion steps of Floyd–Warshall from chapter 2.

From intersections to wards

To colour a ward, we need to know which intersections are inside it. A ward is a polygon, and the test "is this point inside this polygon?" is done with **ray casting** (chapter 12). Then the average time per ward becomes its colour: green is close to care, red is far.

Run it from the reversed graph and the numbers mean "time for an ambulance at this street to reach a hospital", respecting one-way streets.

TRY IT

Close a few roads near a hospital in the Command Center, then reopen the Coverage view. The wards around it turn more orange, because their shortest routes are now longer.

In the code: `server/algorithms/coverage.js`

CHAPTER 9

KD-tree: "what's nearest to this point?"

Every click on the map needs the nearest intersection. Checking all 16,825 each time works, but a KD-tree answers in about 14 steps.

How the tree is built

A 2-d tree is a binary search tree for points on a plane. It splits the points in half again and again, alternating between a vertical cut (by longitude) and a horizontal cut (by latitude).

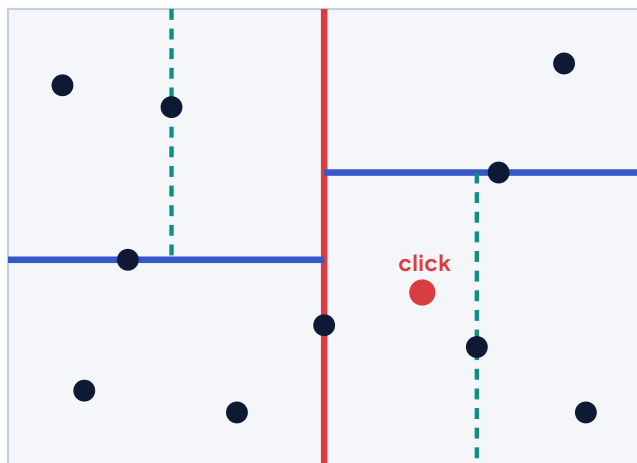


Figure 7. Red: first split. Blue: second level. Green: third level.

1. Find the **median** point by longitude. It becomes the root and splits the map into left and right halves.
2. In each half, find the median by **latitude** and split again.
3. Keep alternating until each region has one point.

Finding a median doesn't need a full sort.

RescueRoute uses **quickselect** (the partition step from quicksort), which is $O(n)$ on average.

Building the whole tree is $O(n \log n)$.

Nearest-neighbour search with pruning

1. Walk down the tree towards the click, like a normal BST search, and remember the closest point seen.
2. On the way back up, ask at each split: could the *other* side hold something closer? Only if the distance from the click to the split line is less than the best distance so far.
3. If not, skip that entire half of the map. This **pruning** is where the speed comes from.

RescueRoute uses KD-trees for three jobs: snapping clicks to the road network, naming each call after the nearest bus stop ("Near Koramangala Water Tank"), and a **radius query** that finds every road within 700 m of a traffic hotspot for rush hour.

In the code: `server/ds/KDTree.js`. The tests compare it with a brute-force scan on 200 random clicks.

CHAPTER 10

Union-find: "are these two places still connected?"

When you close roads, part of the city might get cut off completely. Union-find (also called a disjoint-set union, or DSU) counts the separate pieces almost instantly.

The idea

Every vertex starts in its own group. Each group is a little tree, and the root of the tree is the group's representative.

- **find(x)**: follow parent pointers up to the root. Two vertices are connected if they have the same root.
- **union(a, b)**: find both roots and point one at the other. The two groups become one.

```
function find(x):
    while parent[x] ≠ x:
        parent[x] = parent[parent[x]]    // path halving: skip a level
        x = parent[x]
    return x

function union(a, b):
    ra = find(a); rb = find(b)
    if ra == rb: return
    if size[ra] < size[rb]: swap(ra, rb)    // union by size: small under big
    parent[rb] = ra; size[ra] += size[rb]; components -= 1
```

The two tricks, **path halving** and **union by size**, keep the trees very flat. Each operation costs $O(\alpha(n))$, where α is the inverse Ackermann function. It's below 5 for any input you could ever store, so in practice it is constant time.

Where RescueRoute uses it

1. **After every road closure:** union the ends of every open road, then count components. If the count goes above 1, some area is cut off, and the Network panel shows how many intersections are stranded.
2. **Cleaning the raw data:** bus-route data has tiny disconnected fragments. Union-find finds the largest connected piece and keeps only that, so every intersection can reach every other one.

In the code: `server/ds/UnionFind.js`, used by `Graph.connectivity()`

CHAPTER 11

LRU cache: don't solve the same problem twice

If someone asks for the same route twice, there's no need to search again. A cache remembers recent answers, and "least recently used" decides what to forget when it's full.

Two structures working together

Hash map: key → node. Finds a cached answer in $O(1)$.

Doubly linked list: keeps nodes in order of use, most recent at the front. Moving a node to the front or removing the last one is $O(1)$ because each node knows its neighbours.



when full, drop from the right · on use, move to the left

Figure 8. The linked list keeps the usage order.

- **get(key):** look it up in the map. If found, move its node to the front.
- **set(key, value):** add a node at the front. If the cache is over capacity, remove the node at the back and delete its key from the map.

The clever part: invalidation

A cached route becomes wrong the moment a road closes or traffic changes. Instead of hunting for affected entries, the graph keeps a **version number** that goes up on every change, and the version is part of the cache key: `"from:to:version"`. Old entries simply never match again and eventually fall off the end.

In the code: `server/ds/LRUCache.js`, used by `compareRoutes()` in `Simulation.js`

CHAPTER 12

Small helpers that matter

Not everything is a big algorithm. These three small techniques solve very specific problems neatly.

Binary search: where is the ambulance right now?

When a route is created, `RescueRoute` stores the cumulative travel time at each point: for example `[0, 12, 30, 41, 65, ...]` seconds. To draw the ambulance at time `t`, it needs the first point whose time is at least `t`. The array is sorted, so this is a binary search (a "lower bound"): $O(\log n)$ instead of scanning the route. Then it slides the marker part-way along that segment.

```
function lowerBound(arr, x):
  lo = 0; hi = arr.length
  while lo < hi:
    mid = (lo + hi) / 2
    if arr[mid] < x: lo = mid + 1 else: hi = mid
  return lo
```

Ray casting: is this point inside the ward?

Draw an imaginary ray from the point straight to the right and count how many times it crosses the polygon's edges. **Odd** means inside, **even** means outside. A quick bounding-box check first skips wards that are obviously too far away.

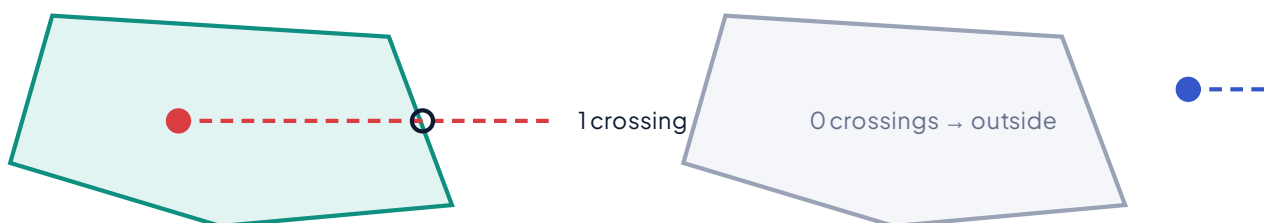


Figure 9. Count crossings: odd inside, even outside.

Douglas–Peucker: fewer points, same shape

The official ward boundaries have thousands of points, too many to draw quickly. Douglas–Peucker is a recursive algorithm: draw a straight line from the first to the last point, find the point farthest from that

line, and if it's more than 25 metres away, keep it and recurse on both halves. Otherwise drop everything in between. It is **divide and conquer**, just like merge sort.

A REAL BUG WE HIT

A ward boundary is a closed loop: its first and last points are the same. The "line from first to last" then has zero length, and every ward collapsed into a single point. The fix was to split each loop at its farthest point first and simplify the two halves separately. Edge cases matter!

CHAPTER 13

Exploring the code yourself

Everything in this guide lives in a small, readable codebase. Here's a map of where to look.

Concept	File	Chapter
Graph model, adjacency lists	<code>server/graph/Graph.js</code>	3
Indexed min-heap	<code>server/ds/IndexedMinHeap.js</code>	4
Triage priority queue	<code>server/ds/PriorityQueue.js</code>	4
Dijkstra, A*, reverse and multi-source search	<code>server/algorithms/search.js</code>	5, 6, 7
Hospital coverage	<code>server/algorithms/coverage.js</code>	8
KD-tree	<code>server/ds/KDTree.js</code>	9
Union-find	<code>server/ds/UnionFind.js</code>	10
LRU cache	<code>server/ds/LRUCache.js</code>	11
Simulation: dispatch, binary search, rerouting	<code>server/sim/Simulation.js</code>	7, 12
Building the Bengaluru map data	<code>scripts/build_bengaluru.py</code>	10, 12
Tests for all of the above	<code>test/</code>	all

Run it on your laptop

```
git clone https://github.com/lakshya0104/dsa.git
cd dsa
npm install
npm start          # open http://localhost:3000
npm test           # runs the 12 correctness tests
```

No database and no build step are needed. You only need Node.js 18 or newer.

Practice exercises

EXERCISES, EASIEST FIRST

1. In `test/ds.test.js`, add a test that pushes 10 numbers into `PriorityQueue` and checks they come out sorted.
2. Change the triage rule so that a serious call waiting more than 10 minutes is treated as critical. Which file and which function would you change?
3. Replace Dijkstra's heap with a plain array scan for the minimum. Time the route planner before and after. What complexity do you get now?
4. Make the A* heuristic twice as large. Does A* still find the optimal route? Why or why not? (Hint: admissibility.)
5. Add a "nearest 3 ambulances" feature using the KD-tree. How would you change `nearest()` to keep the best k instead of the best 1?

Glossary

Term	Meaning
Admissible heuristic	An estimate that never overestimates the true remaining cost. It keeps A* optimal.
Adjacency list	Graph storage where each vertex keeps a list of its edges. Best for sparse graphs like roads.
Complete binary tree	Every level full except possibly the last, filled left to right. This is why a heap fits in an array.
Decrease-key	Lowering the priority of an item already inside a heap, then sifting it up.
Directed graph	Edges have a direction. Needed for one-way streets.
Early exit	Stopping a search as soon as the target is popped, instead of finishing the whole graph.
Heuristic	An informed guess that guides a search, like straight-line distance.
Median split	Dividing points at the middle value so both halves are the same size. Keeps a KD-tree balanced.
Multi-source search	Starting a search from several vertices at once, each at distance 0.
Path compression / halving	Union-find trick: make nodes point closer to the root while finding it.
Pruning	Skipping a part of the search that can't possibly contain a better answer.
Quickselect	Finds the k-th smallest item in $O(n)$ average time using quicksort's partition step.
Relaxation	The Dijkstra step: "is going through u a cheaper way to reach v?" If so, update v.
Sparse graph	A graph with far fewer edges than V^2 . Road networks are very sparse.
Weighted graph	Every edge has a cost. In RescueRoute the cost is travel time in seconds.

ONE-LINE SUMMARY

RescueRoute turns Bengaluru into a weighted directed graph, then uses heaps for priority, Dijkstra and A* for routes, a reverse search for dispatch, a KD-tree for "nearest", union-find for connectivity and an LRU cache for repeats. Each one replaces a slow brute-force loop with something that scales to a whole city.